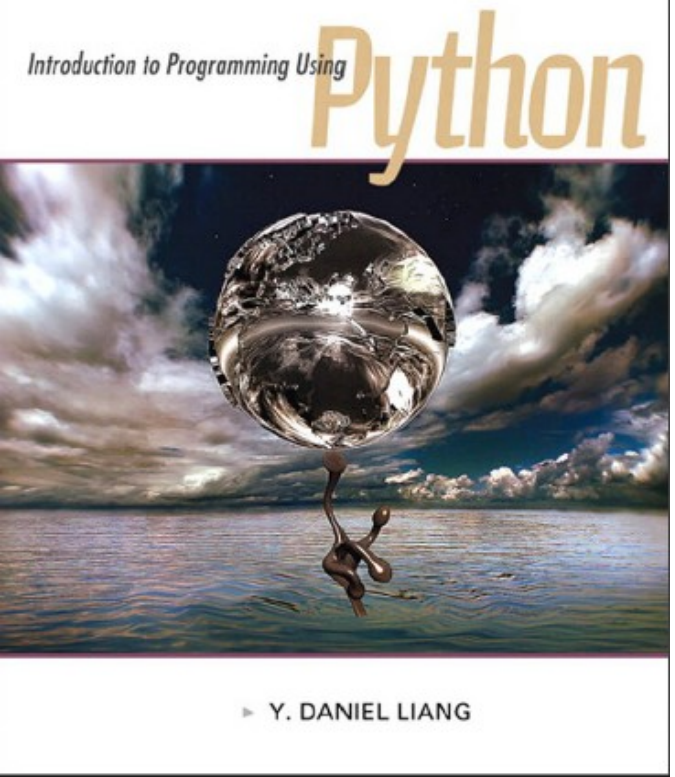



# Chapter 01
# Introduction to Computers, Programs, and Python

Prepared By:

Eng. Eyad J. R. Elfarra

# Objectives

- To understand computer basics, programs, and operating systems ( section 1.2 - 1.4).
- To write and run a simple Python program ( 1.5).
- To explain the basic syntax of a Python program ( 1.5).
- To describe the history of Python ( 1.6).
- To explain the importance of, and provide examples of, proper programming style and documentation ( 1.7).
- To explain the differences between syntax errors, runtime errors, and logic errors ( 1.8).

# Part #1

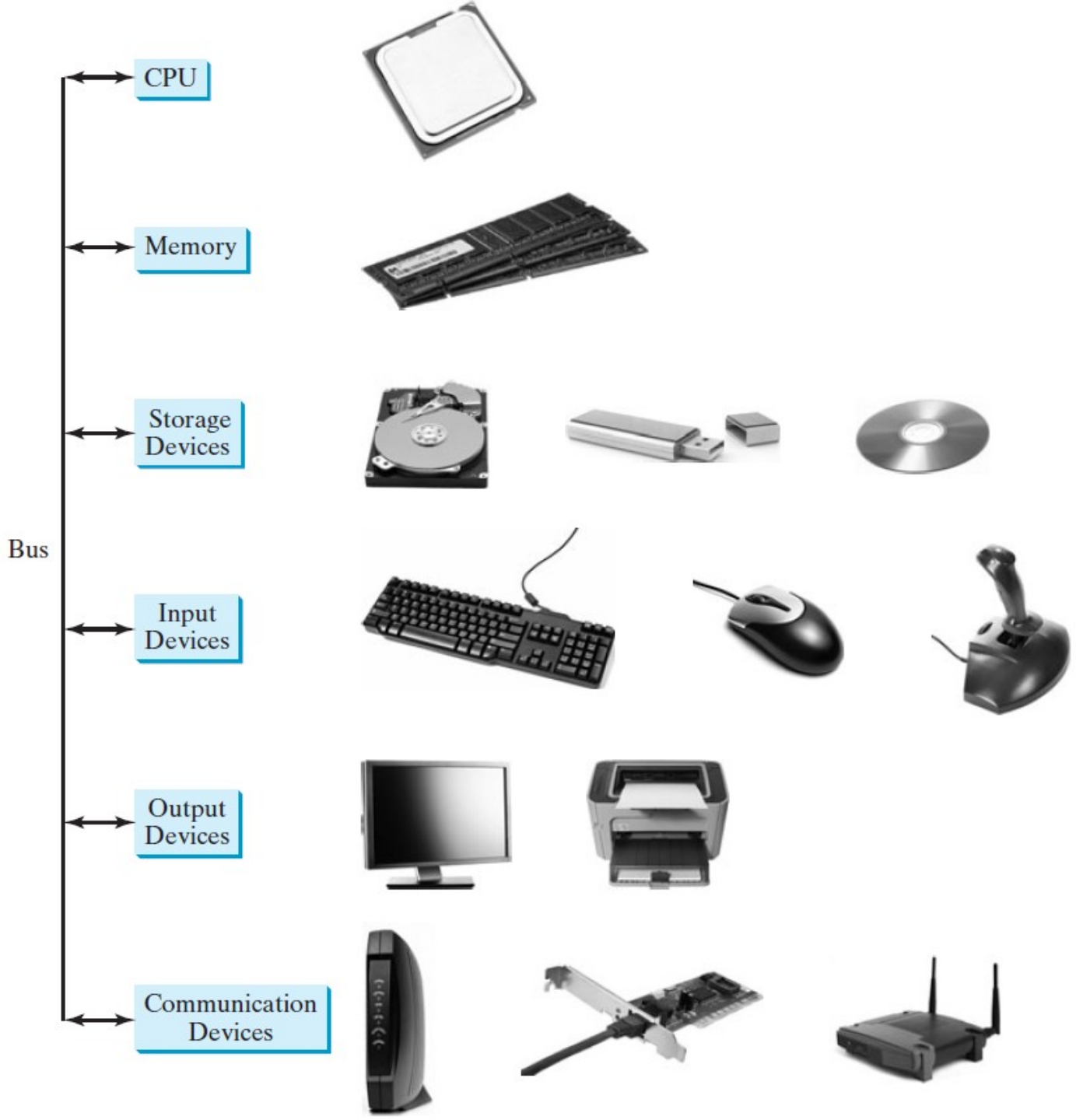
Computer Hardware

# What is  a computer

- A computer is an electronic device that stores and processes data.
- A computer includes:
  - **Hardware**
    - Physical components you can **touch**
  - **Software**
    - Programs and instructions
    - Control how hardware works
- **Hardware** = visible
- **Software** = invisible

# Hardware

- A computer consists of a CPU, memory, hard disk, floppy disk, monitor, printer, and communication devices



# Why Learn Hardware?

- Not required for programming, but:
- Helps understand how programs work
- Shows how instructions affect the computer
- Improves problem-solving skills

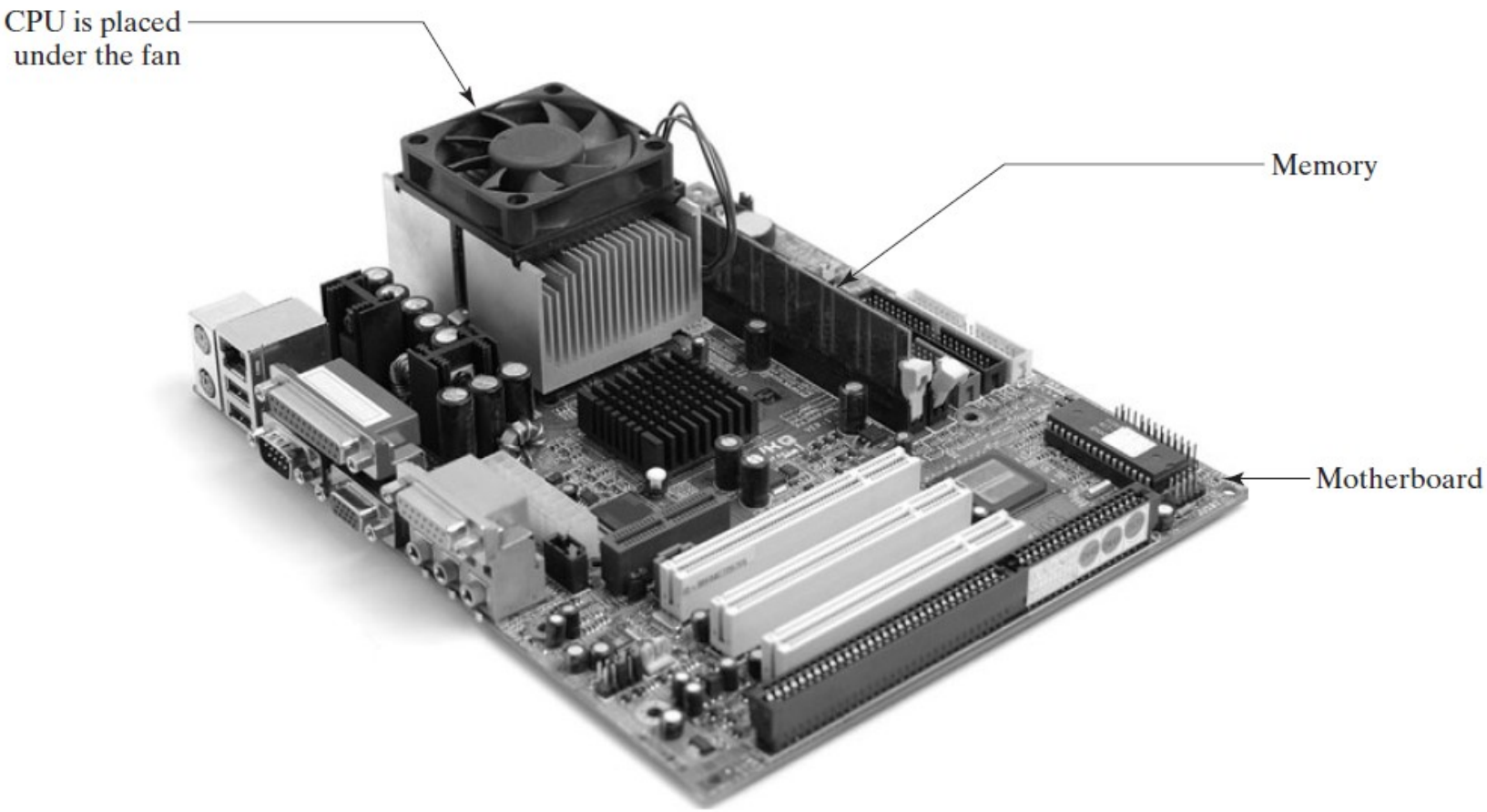
# CPU

- The central processing unit (CPU) is the **brain** of a computer.
- It retrieves instructions from memory and executes them.
- The CPU speed is measured in megahertz (MHz), with 1 megahertz equaling 1 million pulses per second.
  - The speed of the CPU has been improved continuously.
  - If you buy a PC in 2008, you can get an Intel Pentium 4 Processor at 3 gigahertz (1 gigahertz is 1000 megahertz).
  - But now you can get  Core-i9
- Performs calculations
- Controls all operations





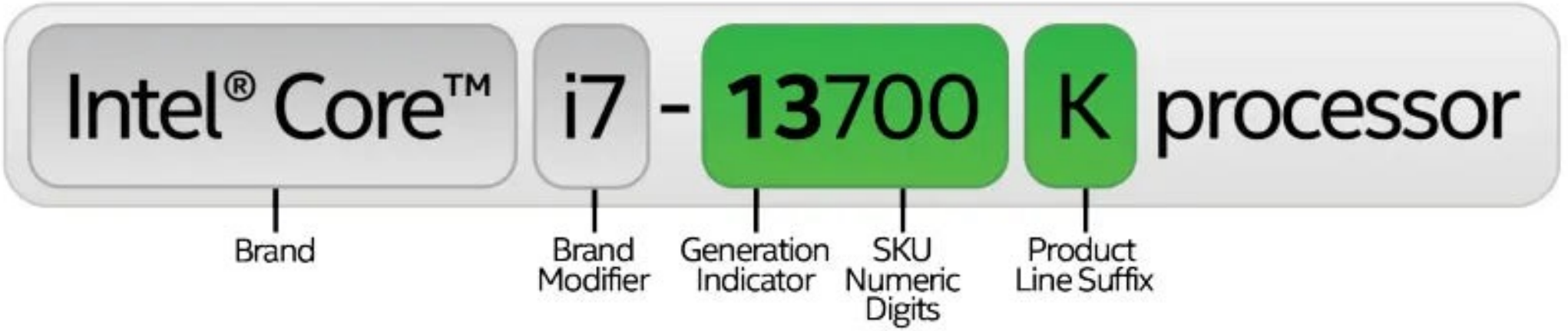

# Where to find CPU?



# CPU Technology

- Built using **silicon semiconductor chips**
- Contains millions of **transistors**
- Transistors act as tiny **switches**
- Enable processing of data
- Early CPUs had one core.
- Modern CPUs have multiple cores.
- **Multicore Processors**
  - Multiple processors in one chip
  - Improves performance and multitasking
  - Examples:
    - Dual-core (2 cores)
    - Quad-core (4 cores)

# CPU


Intel® Core™
i7
-
13700
K
processor
Brand
Brand Modifier
Generation Indicator
SKU Numeric Digits
Product Line Suffix

# Bits and Bytes

- Computers store data using **binary system**
- Everything is represented using:
  - **0 (off)**
  - **1 (on)**
- **What is a Bit?**
  - Bit = **Binary Digit**
  - Smallest unit of data
  - Can be:
    - 0 → OFF
    - 1 → ON
  - Represents a switch state

# What is a Byte?

- A **byte = 8 bits**
- The minimum practical storage unit in the computer.
- **No two data** can **share** or **split** a same byte.
- Can store:
  - Numbers
  - Characters
- Example:
  - Number 3 stored in one byte
- Large values require **multiple bytes**

# Data Encoding

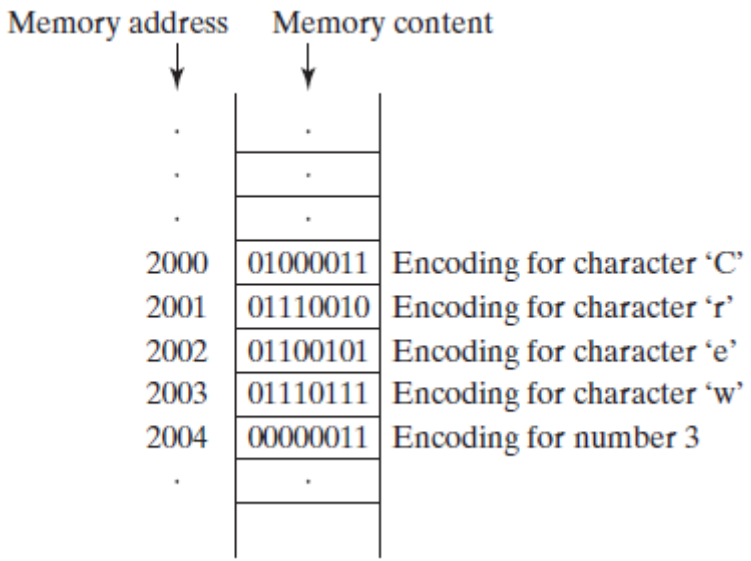
- Data stored as **series of bytes**
- Encoding = converting data into binary
- Done automatically by the system
- **ASCII**
  - Popular encoding scheme
  - Example:
    - Character C → 01000011
  - Each character stored in 1 byte

# Storage Units

- **Kilobyte (KB)** ≈ 1,000 bytes
- **Megabyte (MB)** ≈ 1,000,000 bytes
- **Gigabyte (GB)** ≈ 1,000,000,000 bytes
- **Terabyte (TB)** ≈ 1,000,000,000,000 bytes

# Memory (Main Memory)

- Stores data temporarily.
- Used while programs are running:
  - Store **data** and **program instructions** for CPU to execute.
- Also called **RAM**
- Fast but temporary (volatile)
- A memory unit is an ordered sequence of bytes, each holds eight bits.

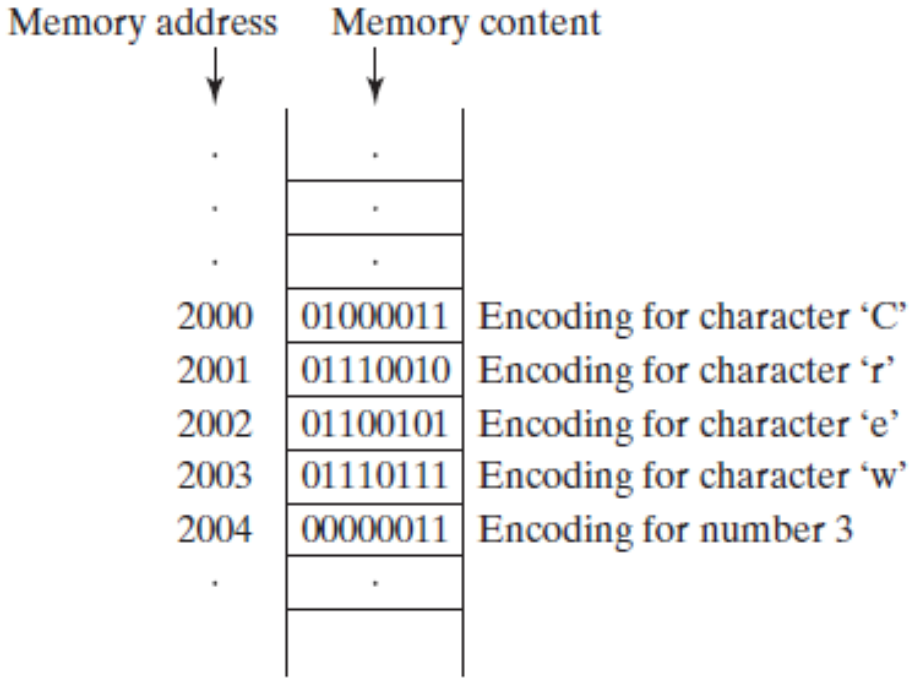


# Memory (Main Memory)

- A program and its data must be brought to memory *before* they can be executed.
- A memory byte is never empty, but its initial content may be **meaningless** to your program.
- The current content of a memory byte is lost whenever new information is placed in it.

# RAM

- **Why Memory is Important**
  - Programs must be loaded into memory
  - Data must be in memory to be processed
  - CPU **cannot execute** without memory
- **Random Access Memory (RAM)**
  - Memory is called **RAM**
  - "Random access" means:
    - Any byte can be accessed directly
  - No need to follow a sequence

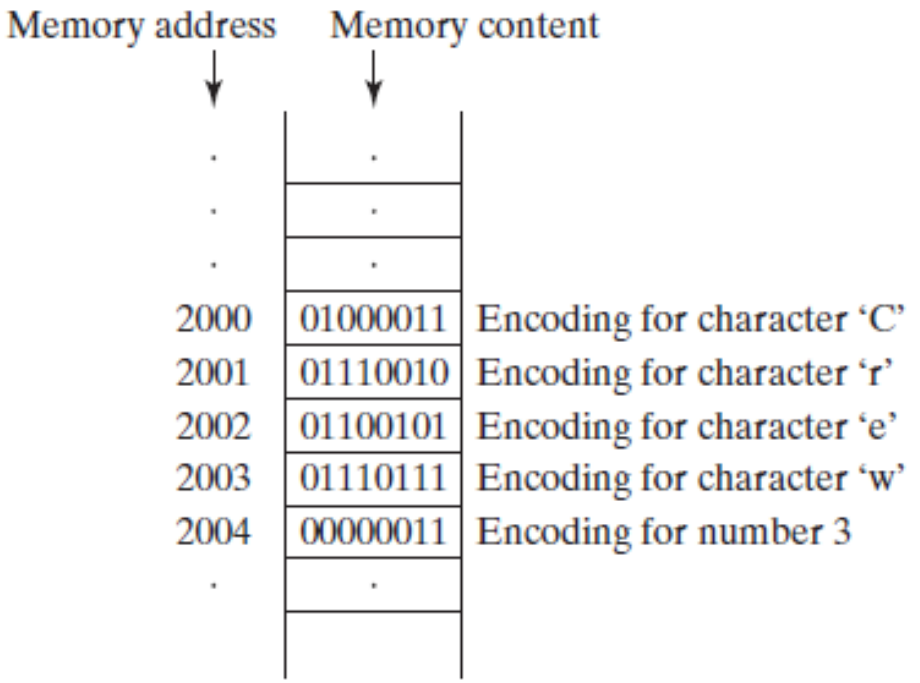


# RAM Capacity & Technology

- **RAM Capacity**
  - Typical modern computers:
    - At least **1 GB RAM**
    - Common: **2–4 GB or more**
  - More RAM → generally better performance
- **Memory Technology**
  - Built using **silicon chips**
  - Contains millions of **transistors**
  - Compared to CPU:
    - Simpler
    - Slower
    - Cheaper

# Memory Addresses

- Every byte has a **unique address**
- Address used to:
  - Store data
  - Retrieve data
- Works like **house addresses**



# Storage Devices

- Used for **permanent data storage**
- Stores:
  - Programs
  - Files
- Data remains even when power is OFF
- There are three main types:
  - Magnetic Disk Drives
  - Optical Disc Drives (CD/DVD)
  - Flash Drives:
    - USB
    - SSD

# Hard Disk Drives

- Main storage device in computers
- Stores large amounts of data.
- There is a **moving arm** and head to read the data.
- Capacity:
  - Typically **500–2048 GB or more**
- Can be:
  - Internal
  - External (removable)



# Solid State Device(SSD) Storage

- Sometimes called flash storage
- Set of interconnected flash memory cards
- No moving parts
- More costly than HDDs
- DRAM-based SSDs available
  - Faster access times compared with flash

# Solid State Device Storage



# USB Flash Drives

- Small, portable storage devices
- Connect via **USB port**
- Easy to carry and use
- USB Features
  - Acts like a portable hard drive
  - Used for:
    - Transferring files
    - Backup
  - Capacity:
    - Up to 1 TB





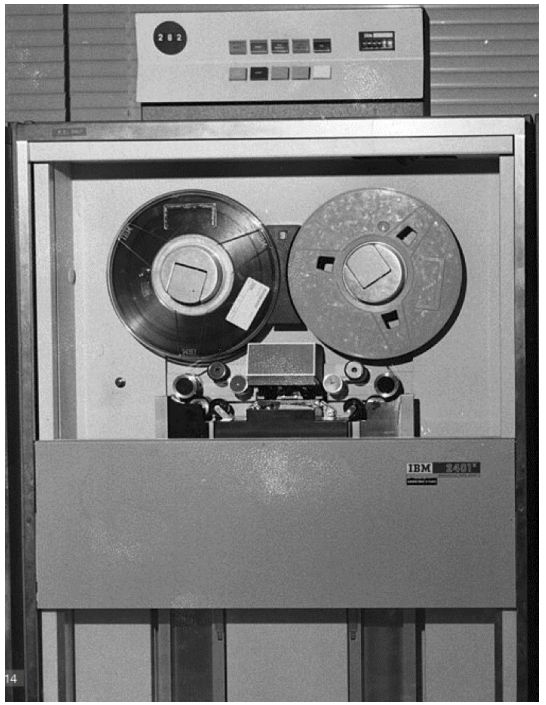

# Magnetic Tape Storage Devices

- Sequential access
  - Must scan preceding blocks

- Tape is mounted and scanned until required block is under read/write head

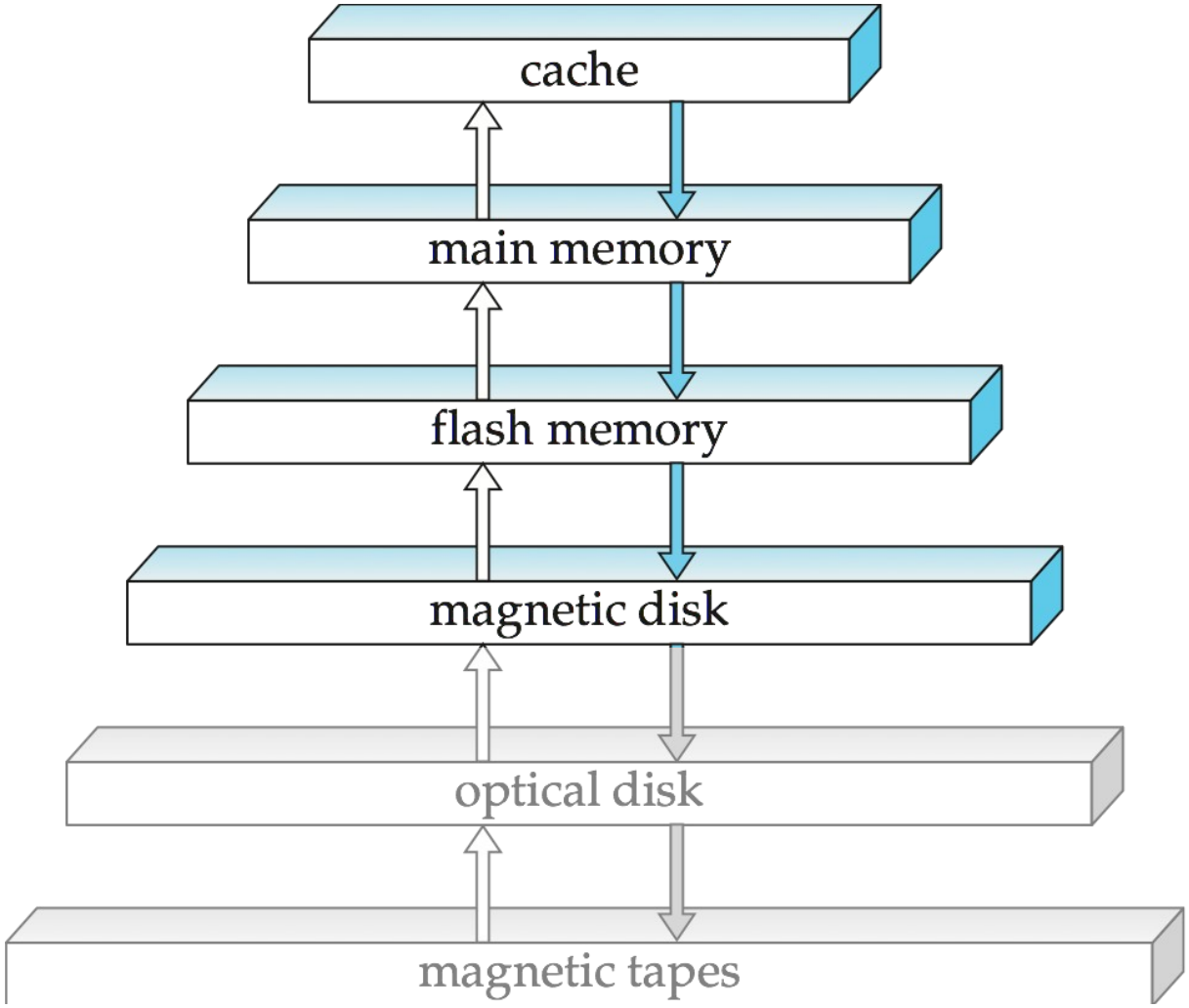
- Important functions
  - Backup
  - Archive





# Memory Hierarchies and Storage Devices



# Storage Types and Characteristics

| Type | Capacity* | Access Time | Max Bandwidth |
|---|---|---|---|
| Main Memory- RAM | 4GB–1TB | 30ns | 35GB/sec |
| Flash Memory- SSD | 64 GB–1TB | 50µs | 750MB/sec |
| Flash Memory- USB stick | 4GB–512GB | 100µs | 50MB/sec |
| Magnetic Disk | 400 GB–8TB | 10ms | 200MB/sec |
| Optical Storage | 50GB–100GB | 180ms | 72MB/sec |
| Magnetic Tape | 2.5TB–8.5TB | 10s–80s | 40–250MB/sec |
| Tape jukebox | 25TB–2,100,000TB | 10s–80s | 250MB/sec–1.2PB/sec |

# Memory vs Storage

- **Memory (RAM)**
  - Temporary (volatile)
  - Fast
  - Lost when power is off
- **Storage Devices**
  - Permanent
  - Slower than memory
  - Keeps data safely

# Input Devices

- I/O devices allow users **to communicate with the computer.**
- Used to **enter data into the computer**
- Examples:
  - Keyboard
  - Mouse
  - Scanner



# Output Devices

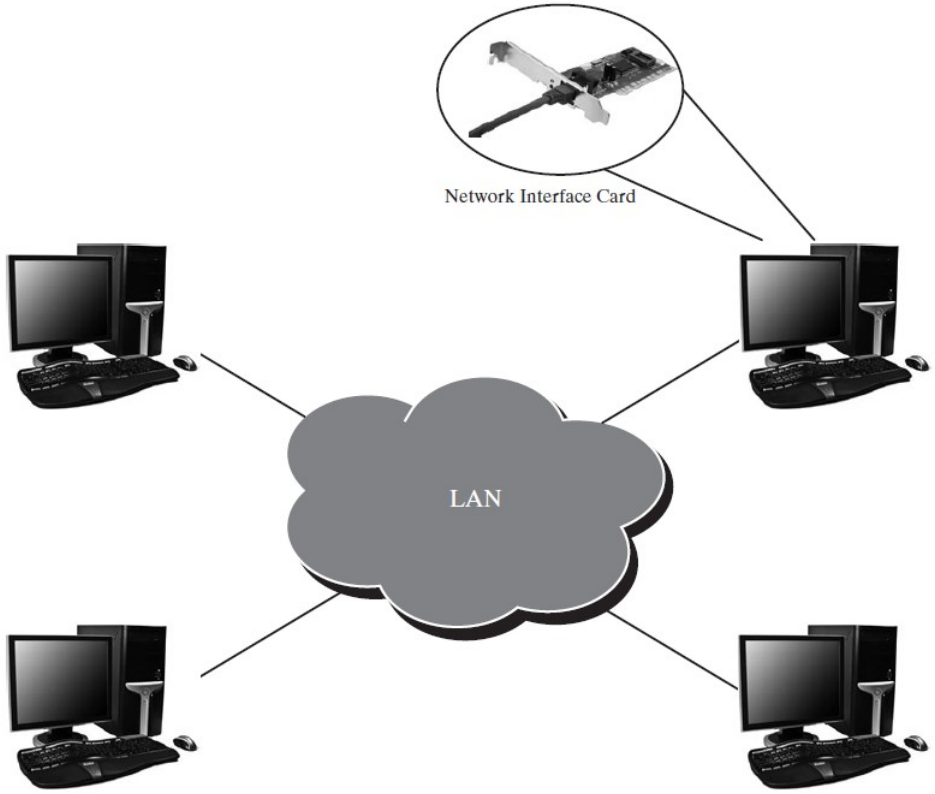
- Used to **display results**
- Examples:
  - Monitor
  - Printer
  - Speakers

# The Monitor

- Main output device
- **Screen Resolution**
  - Number of pixels on screen
  - Example:
    - 1024 × 768
  - Higher resolution → better clarity
- **Pixels**
  - Small dots forming the display
  - More pixels → sharper image

# Communication Devices

- Enable computers to connect and communicate
- Used for:
  - Internet access
  - Data transfer
  - Networking
- Examples:
  - Wireless Adapter
    - Enables Wi-Fi connection
    - No cables needed
    - Built into most laptops
  - Network Interface Card (NIC)
    - Connects computer to **local network (LAN)**

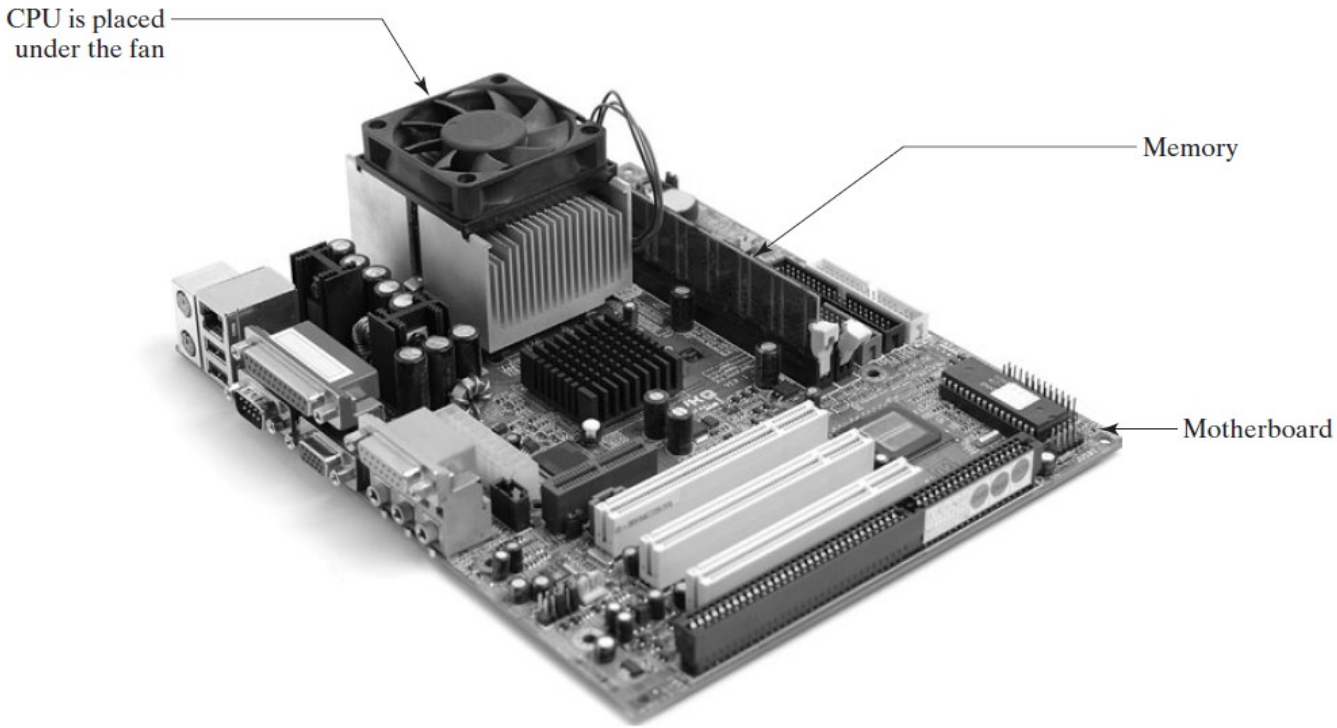


# The Bus System

- A **bus** connects all components
- Transfers:
  - Data
  - Signals
- Works like a **road system** inside the computer

# Motherboard

- Main circuit board of the computer
- Connects all components together
- Contains the bus system



# Part# 02

Programming Languages & Operating Systems

# Programs

- Programs , known as software or apps , are instructions to the computer.
- You tell a computer what to do through programs.
- **Without** programs, a computer is an **empty machine**.
- Computers do not understand human languages, so you need to use computer languages to communicate with them.
- Programs are written using **programming languages**.

# Programming Languages

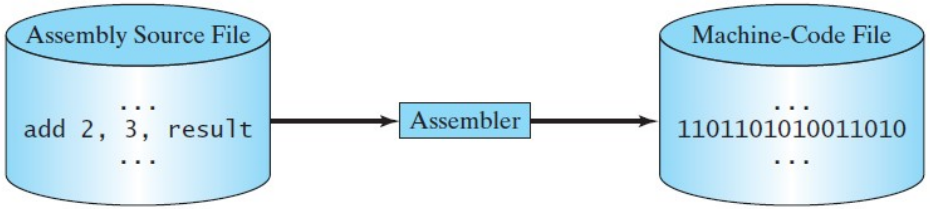
- **Why Programming Languages?**
  - Make programming easier for humans
  - Allow writing readable instructions
  - Must be translated into **machine language**
- **Types of Programming Languages:**
  - Machine Language
  - Assembly Language
  - High-Level Language

# Machine Language

- Native language of the computer
- Written in **binary (0s and 1s)**
- Example:
  - 1101101010011010
- Very fast but difficult to use
- **Characteristics of Machine Language**
  - Machine-dependent
  - Hard to read and write
  - Error-prone
  - Used directly by CPU

# Assembly Language

- Assembly languages were developed to make programming easy.
- Since the computer **cannot** understand assembly language
  - A program called **assembler** is used to convert assembly language programs into machine code.
- For example, to add two numbers, you might write an instruction in assembly code like this:
  - **ADDF3 R1, R2, R3**



# Low-Level Language

- Assembly language is referred to as **a low-level language**
- Close to machine language
- Requires knowledge of hardware(**machine dependent)**

# High Level Language

- The high level languages are English like and easy to learn and program.
- Platform-independent
- For example, the following is a high level language statement that computes the area of a circle with
- radius 5:
  - area = 5 * 5 * 3.1415;

# High-Level Languages

- **Features of High-Level Languages**
  - Easy to learn
  - Portable (run on different machines)
  - Faster development
  - Used in modern programming
- **Examples of High-Level Languages**
  - Python
  - Java
  - C / C++
  - C#
  - Visual Basic
  - FORTRAN
  - Pascal

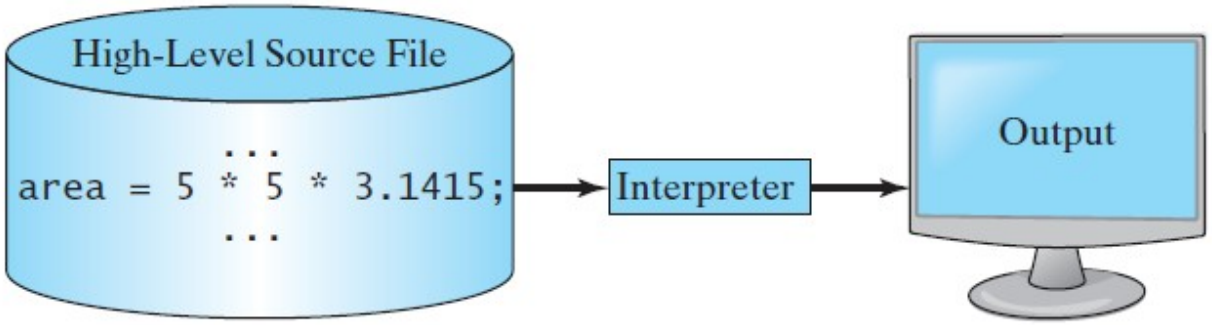


# High-Level Languages

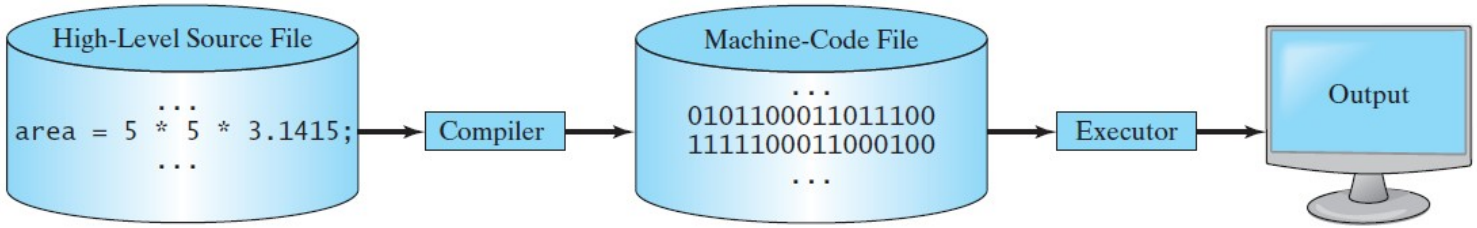
- **Source Code**
  - Program written by programmer
  - Also called:
    - **Source program**
  - Must be translated to machine code
- **Translator Programs**
  - Two types:
    - Interpreter
    - Compiler

# Interpreter vs Compiler

- **Interpreter**
  - Translates **one line at a time**
  - Executes immediately
  - Slower but easier to debug
  - Example: Python

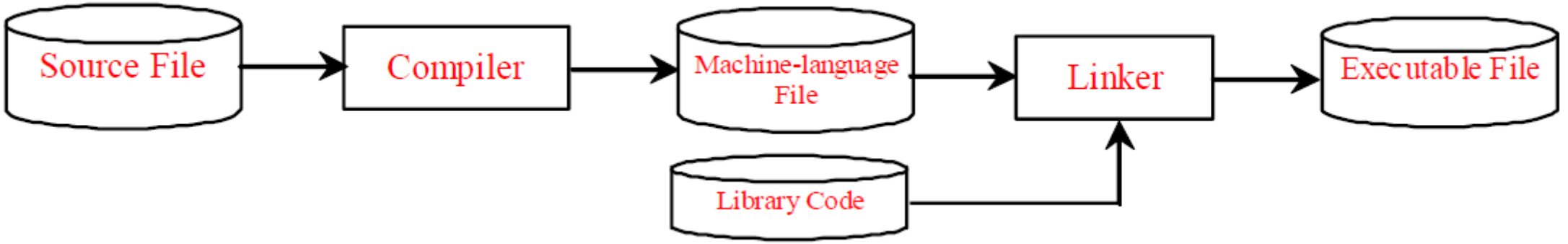



- **Compiler**
  - Translates entire program at once
  - Produces machine code file
  - Faster execution
  - Used in languages like C++



# Compiling Source Code

- A program written in a high level language is called a **source program**.
- Since a computer cannot understand a source program. Program called a **compiler** is used to **translate** the **source program** into a **machine language** program called an **object program** .
- The object program is often then linked with other supporting library code before the object can be executed on the machine.

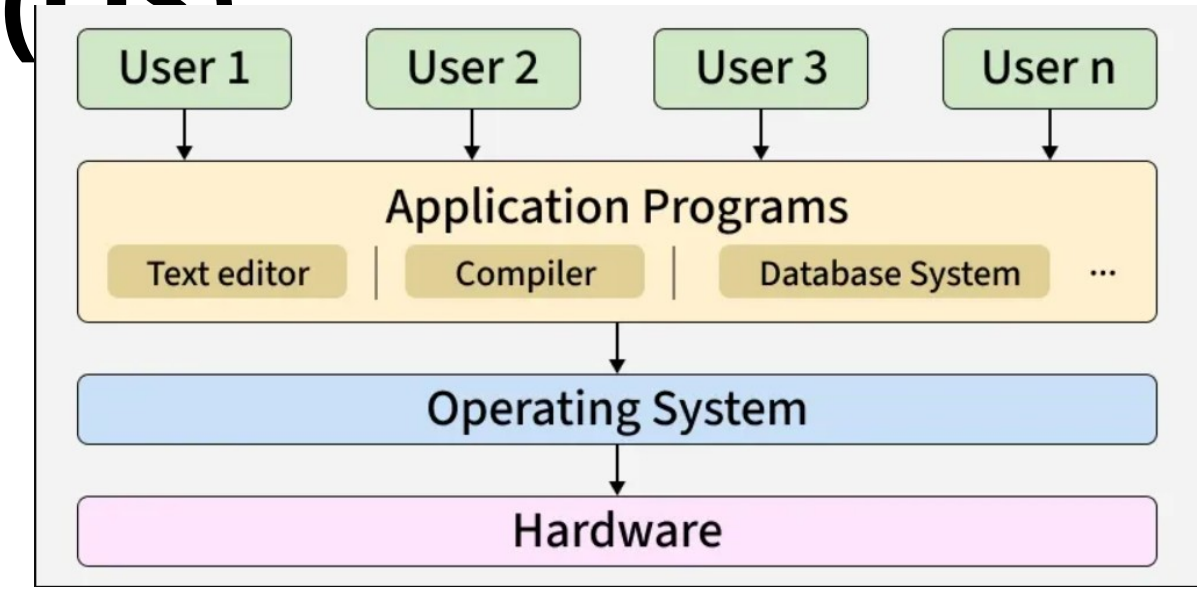


# Operating Systems

- The operating system (OS) is the most important program that runs on a computer.
- The OS manages and controls a computer's activities.



# Operating System (OS)




- Connects:
  - User
  - Applications
  - Hardware
- Controls computer operations
- The major tasks of an operating system are:
  - Controlling and monitoring system activities
  - Allocating and assigning system resources
  - Scheduling operations

# Controlling and Monitoring System Activities

- Operating systems perform **basic tasks**, such as:
  - **Recognizing** input from the ***keyboard***,
  - **Sending** output to the ***monitor***,
  - **Keeping track** of *files and folders* on ***storage devices***,
  - **Controlling** peripheral devices, such as ***disk drives and printers***.
- An operating system must also ensure that different programs and users **working at the same time** do not interfere with each other.
- The OS is responsible for **security**, ensuring that **unauthorized** users and programs do not access the system.

# Allocating and Assigning System Resources

- The operating system is responsible:
  - For **determining** what computer **resources** a program needs, such as
    - CPU time.
    - Memory space.
    - Disks.
    - Input and output devices.
  - For **allocating** and **assigning** them to run the program.

# Scheduling Operations

- The OS is responsible for scheduling programs' activities to make efficient use of system resources.
- OS supports *multiprogramming*, *multithreading*, and *multiprocessing* to increase system performance.

# Multiprogramming

- Multiprogramming allows **multiple programs** to run simultaneously by **sharing** the same CPU.
- The CPU is **idle** most of the time.
  - Because it is **faster** than any other computer component.
- For example, ***multiprogramming*** enables you to use a **word processor** to edit a file at the same time as your **Web browser** is downloading a file.

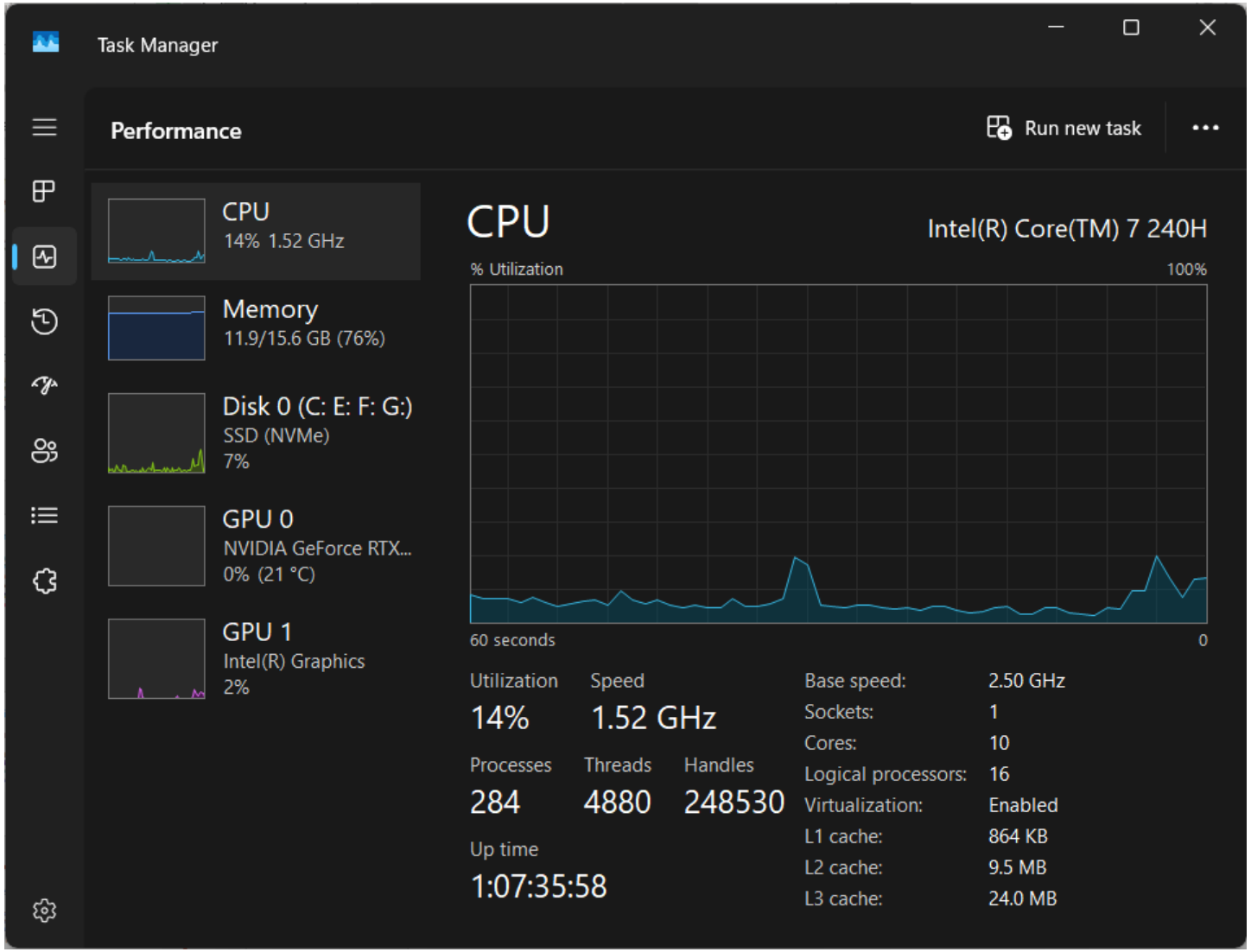
# Multithreading

- Multithreading allows a **single program** to execute **multiple tasks** at **the same time**.
- A word-processing program allows users to simultaneously **edit text** and **save it to a disk**.
  - In this example, editing and saving are two tasks within the same application.
  - These two tasks may run concurrently.

# Multiprocessing

- Multiprocessing, or **parallel processing**, uses **two or more processors** together to perform **subtasks concurrently** and then **combine** solutions of the subtasks to obtain a solution for the entire task.
- It is like a surgical operation where several doctors work together on one patient.
- As done in AI.

# OS Task Manager



# Part #3

Python

# What is Python?

- General Purpose
  - Python is a general purpose programming language.
  - That means you can use Python to write code for any programming tasks.
  - Python are now used in
    - Google search engine.
    - Mission critical projects in NASA.
    - Processing financial transactions at New York Stock E
    - Also it used in AI and data science research.
    - Hardware programming.
- Interpreted
- Object Oriented



# What is Python?

- General Purpose
- Interpreted
  - Python is interpreted, which means that python code is translated and executed by an interpreter one statement at a time.
  - In a compiled language, the entire source code is compiled and then executed altogether.
- Object Oriented

# What is Python?

- General Purpose
- Interpreted
- Object Oriented
  - Python is an object oriented programming language.
  - Data in Python are objects created from classes.
  - A class is essentially a type that defines the objects of the same kind with properties and methods for manipulating objects.
  - Object oriented programming is a powerful tool for developing reusable software.

# Python

- Created by Guido van Rossum in Netherlands in 1990
- **Open source.**
- Named after the British comedy group, **Monty Python**






# Python Popularity

- **TIOBE Index for March 2026**

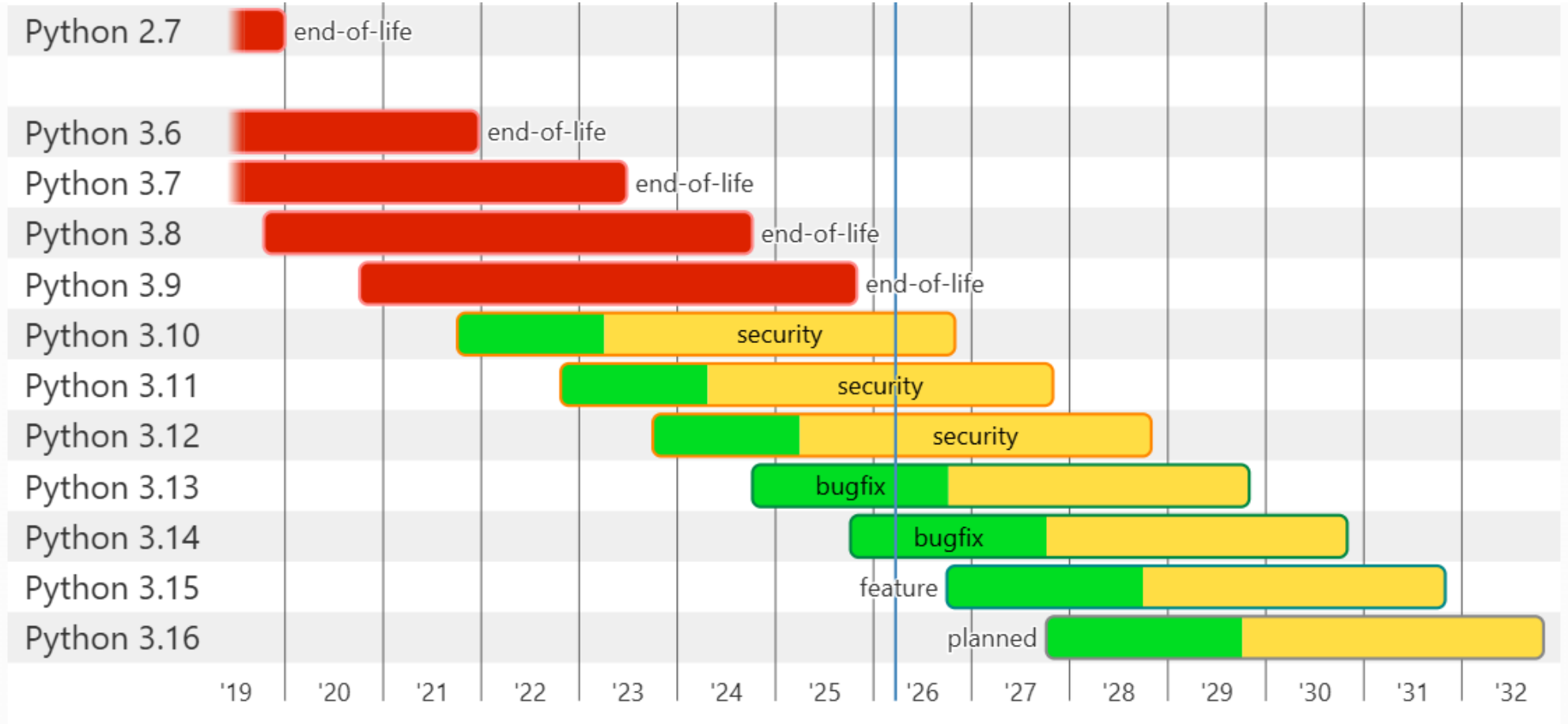
| Mar 2026 | Mar 2025 | Change | Programming Language | Ratings | Change |
|---|---|---|---|---|---|
| 1 | 1 | | Python | 21.25% | -2.59% |
| 2 | 4 | ↑ | C | 11.55% | +2.02% |
| **3** | **2** | ↓ | **C++** | **8.18%** | **-2.90%** |
| 4 | 3 | ↓ | Java | 7.99% | -2.37% |
| 5 | 5 | | C# | 6.36% | +1.49% |
| 6 | 6 | | JavaScript | 3.45% | -0.01% |
| 7 | 9 | ↑ | Visual Basic | 2.50% | -0.02% |
| 8 | 8 | | SQL | 2.00% | -0.57% |
| 9 | 16 | ⇈ | R | 1.88% | +0.94% |

# Python 2 vs. Python 3

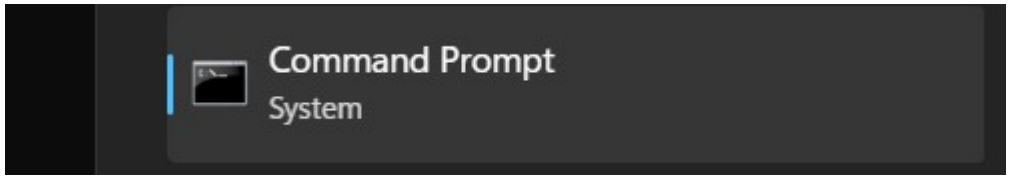
- Python 3 is a newer version, but it is not backward compatible with Python 2.
  - That means if you write a program using Python 2, it may not work on Python 3.

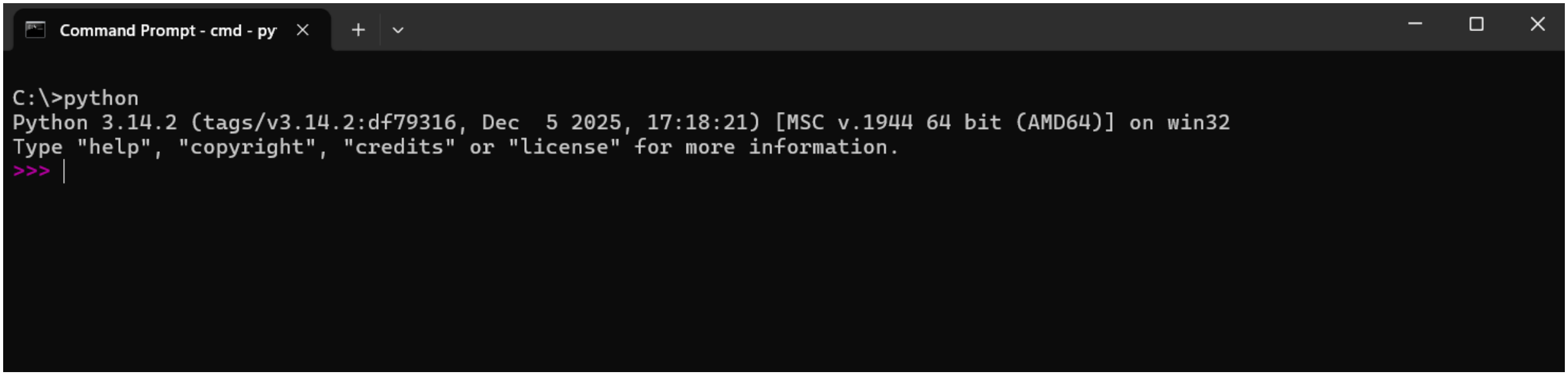
- Current Version of python 3.14

# Python Versions

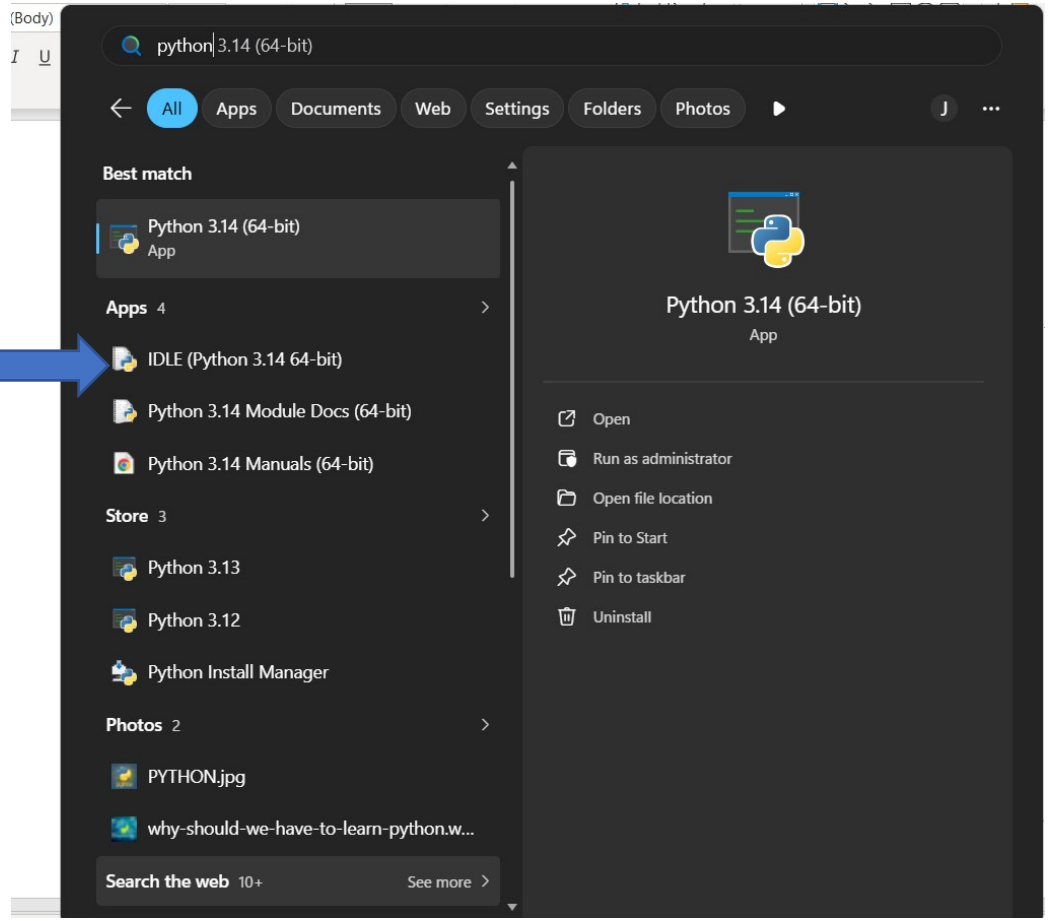


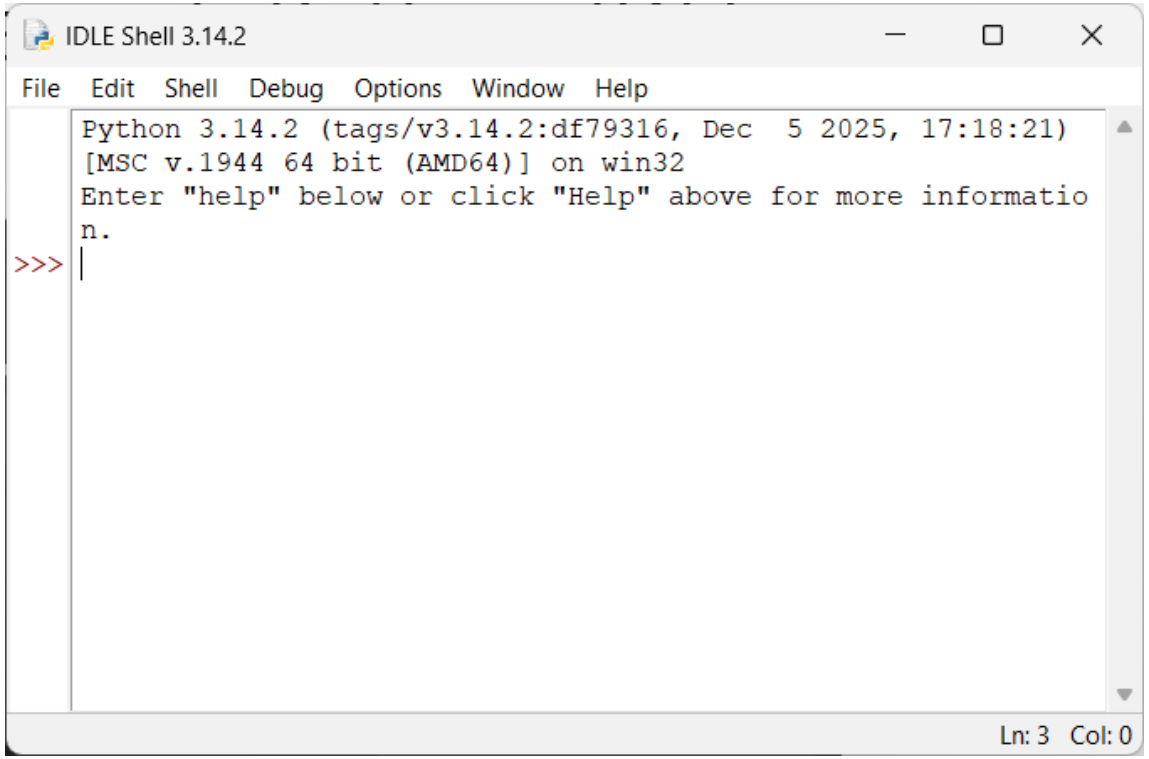
# Launch Python



# Launch Python **IDLE** (Integrated Development and Learning Environment)

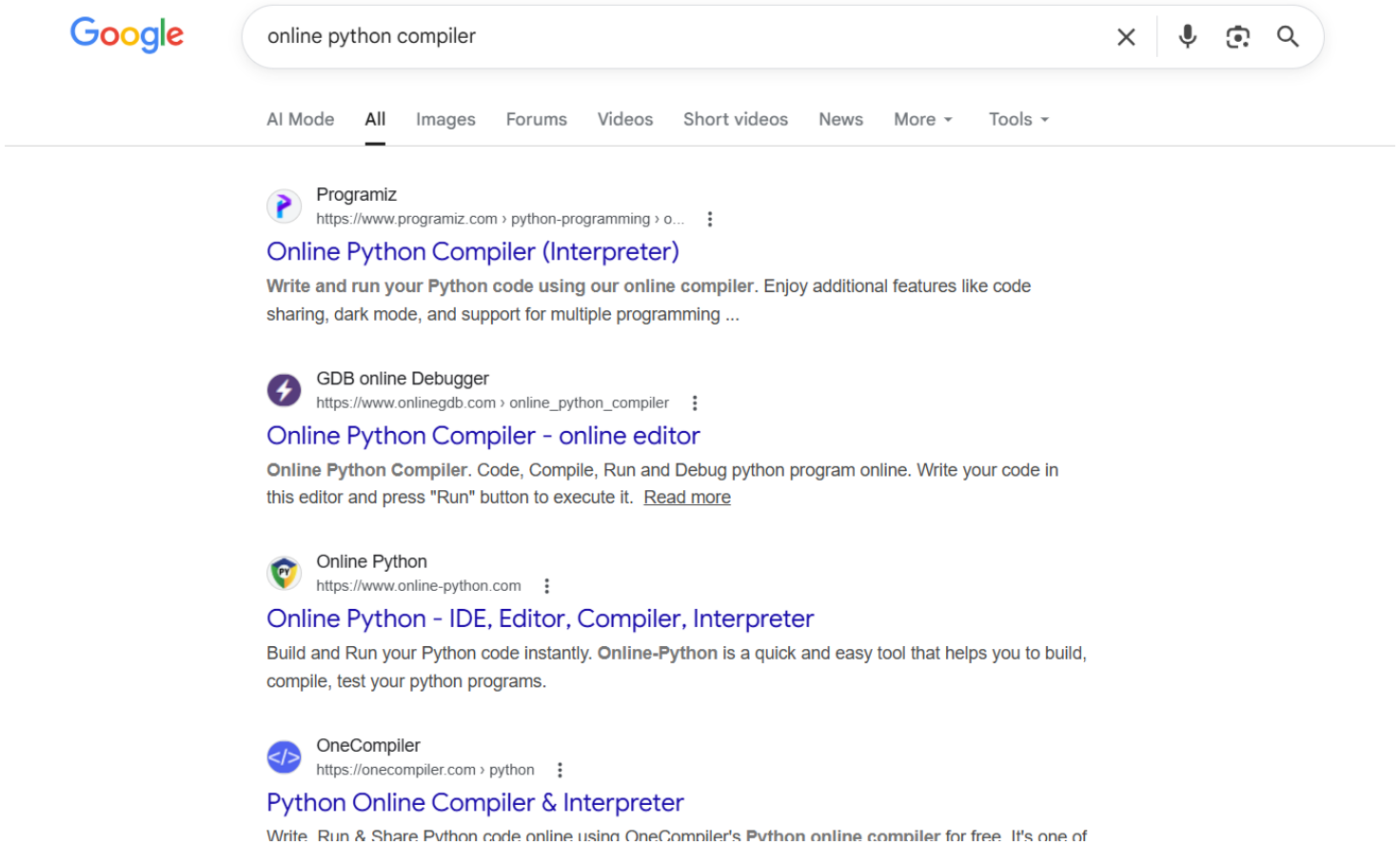


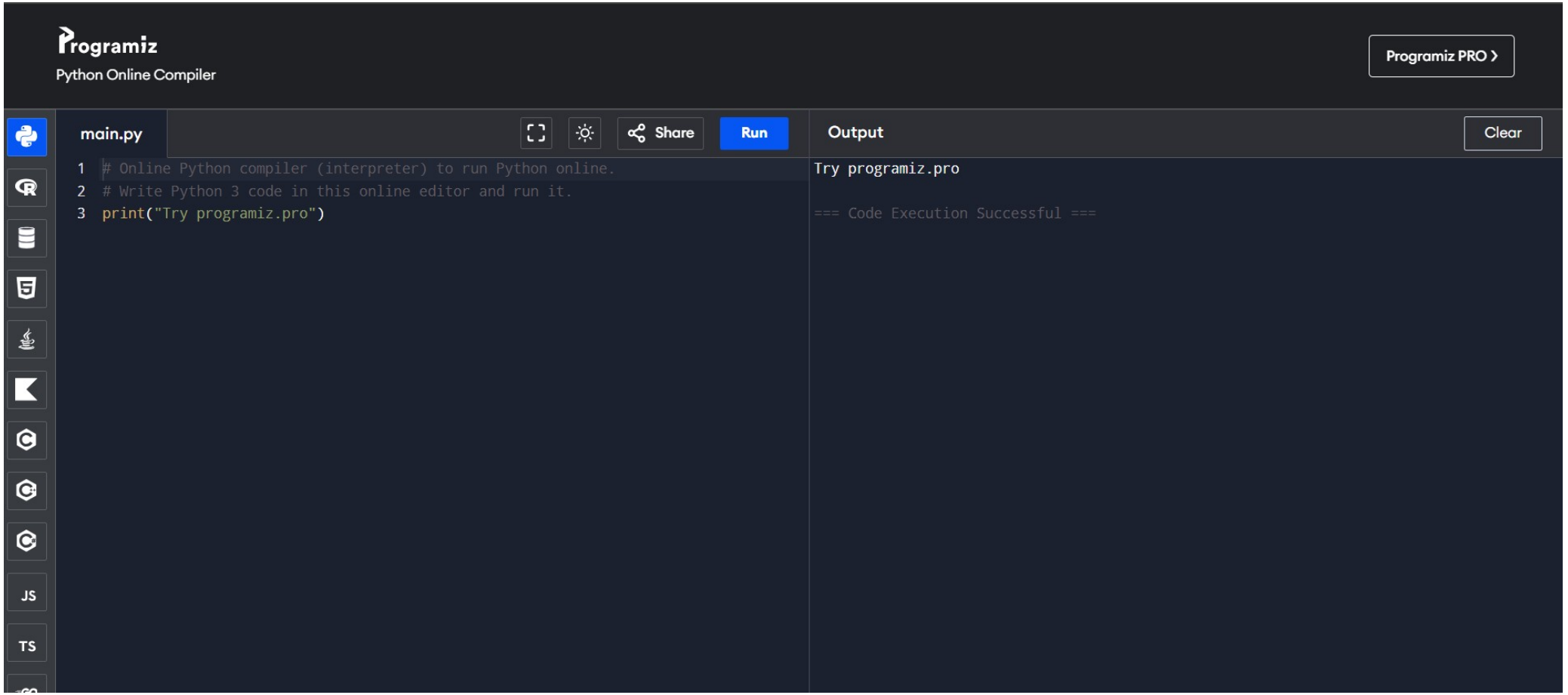
# Programming Python online

- You can write and execute python scripts without install interpreter in your device.
- You can use the online services such as:
  - **W3schools** python course online interpreter.
  - **programiz** python online compiler.
  - Also **pydroid3** for mobile.

# Online Python Compiler



# Programiz


Programiz
Python Online Compiler
Programiz PRO ›
main.py
Share
Run
Output
Clear
1 # Online Python compiler (interpreter) to run Python online.
2 # Write Python 3 code in this online editor and run it.
3 print("Try programiz.pro")
Try programiz.pro
=== Code Execution Successful ===
JS
TS

# Talking to Python

```
>>> x = 1
>>> print(x)
1
>>> x = x + 1
>>> print(x)
2
>>> exit()
```

This is a good test to make sure that you have Python correctly installed. Note that quit() also works to end the interactive session.

# Run Python Script

- Open file
- Write the following lines

```
# Display two messages
print("Welcome to Programming Fundamentals
Class")
print("Python is fun")
```

- Save file as “welcome.py”
- Then open the command line
  - Navigate to the file location
  - Then type python  welcome.py

# Programming Paragraphs

# Interactive versus Script

- **Interactive**

  - You type directly to Python one line at a time and it responds

- **Script**

  - You enter a sequence of statements (lines) into a file using a text editor and tell Python to execute the statements in the file

# Python Scripts

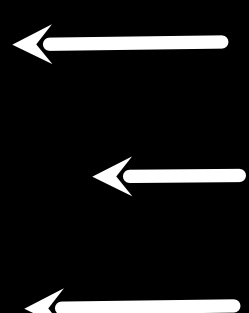
- **Interactive** Python is good for experiments and programs of **3-4 lines** long.
- Most programs are much **longer**, so we type them into a **file** and tell Python to **run** the commands in the file.
- As a convention, we add "**.py**" as the **suffix** on the end of these files to indicate they contain Python.

# Anatomy of a Python Program

- Statements –sentences - lines
- Comments
- Indentation

# Sentences or Lines

```
x = 2
x = x + 2
print(x)
```



Assignment statement

Assignment with expression

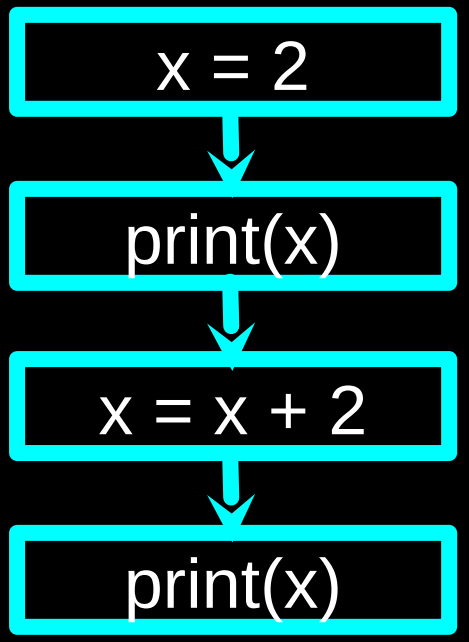
Print statement

Variable Operator Constant Function

# Statements

- A statement represents an action or a sequence of actions.
- Like a recipe or installation instructions, a program is a sequence of steps to be done in order.
- Some steps are conditional - they may be skipped.
- Sometimes a step or group of steps is to be repeated.
- Sometimes we store a set of steps to be used over and over as needed several places throughout the program. → functions

# Sequential Steps



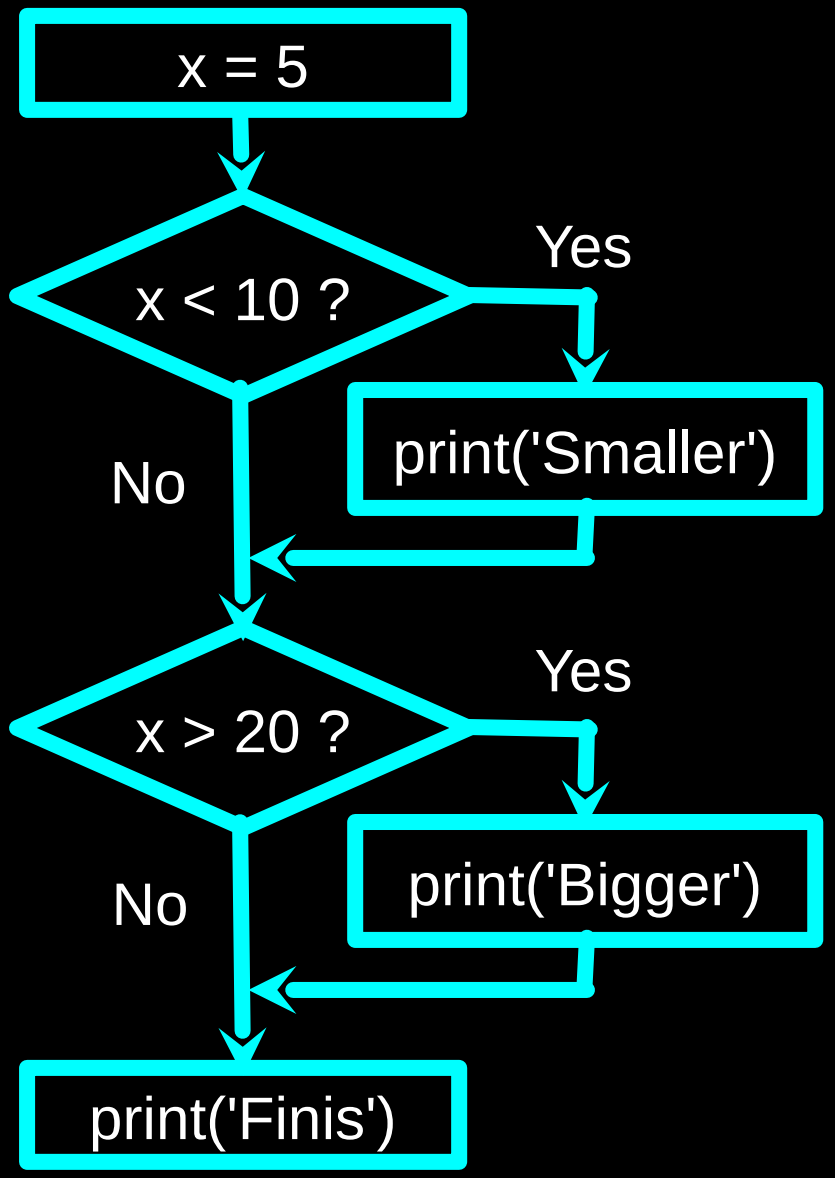

Program:

```
x = 2
print(x)
x = x + 2
print(x)
```

Output:

```
2
4
```

When a program is running, it flows from one step to the next. As programmers, we set up “paths” for the program to follow.




# Conditional Steps

Program:

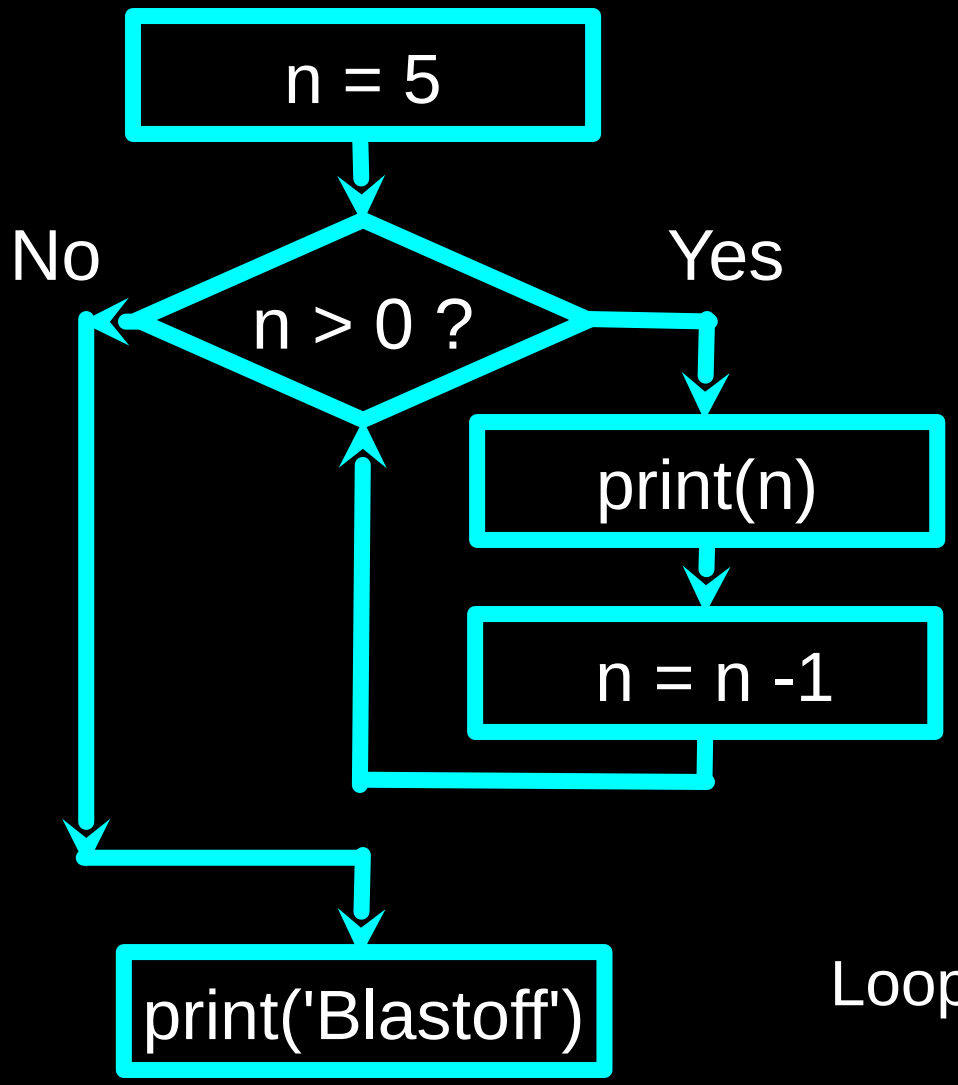
```
x = 5
if x < 10:
    print('Smaller')
if x > 20:
    print('Bigger')

print('Finis')
```

Output:

```
Smaller
Finis
```

# Repeated Steps




Program:

```
n = 5
while n > 0 :
    print(n)
    n = n - 1
print('Blastoff!')
```

Output:

```
5
4
3
2
1
Blastoff!
```

Loops (repeated steps) have iteration variables that change each time through a loop.

# Indentation

- The indentation matters in Python.
- Note that the statements are entered from the **first column** in the new line.

```
# Display two messages
print("Welcome to Python")
print("Python is fun")
```

- It would **cause an error** if the program is typed as follows:

```
# Display two messages
  print("Welcome to Python")
print("Python is fun")
```

X

# Special Symbols

| *Character* | *Name* | *Description* |
|---|---|---|
| () | Opening and closing parentheses | Used with functions. |
| # | Pound sign | Precedes a comment line. |
| " " | Opening and closing quotation marks | Encloses a string (i.e., sequence of characters). |
| ''' ''' | Paragraph comments | Encloses a paragraph comment. |

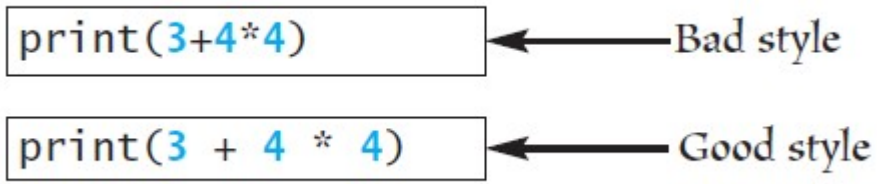
# Programming Style and Documentation

- Appropriate Comments
  - Include a **summary** at the beginning of the program to **explain** what the program does, its key features, its supporting data structures, and any unique techniques it uses.
- Proper Indentation and Spacing Lines

# Proper Indentation and Spacing

- Indentation
  - Indent four spaces.
  - A consistent spacing style makes programs clear and easy to read, debug, and maintain.

- Spacing
  - Use blank line to separate segments of the code
  - surround functions with 2 blank lines



# Programming Errors

- Errors are common in programming
- Three main types:
  - **Syntax** Errors
  - **Runtime** Errors
  - **Logic** Errors

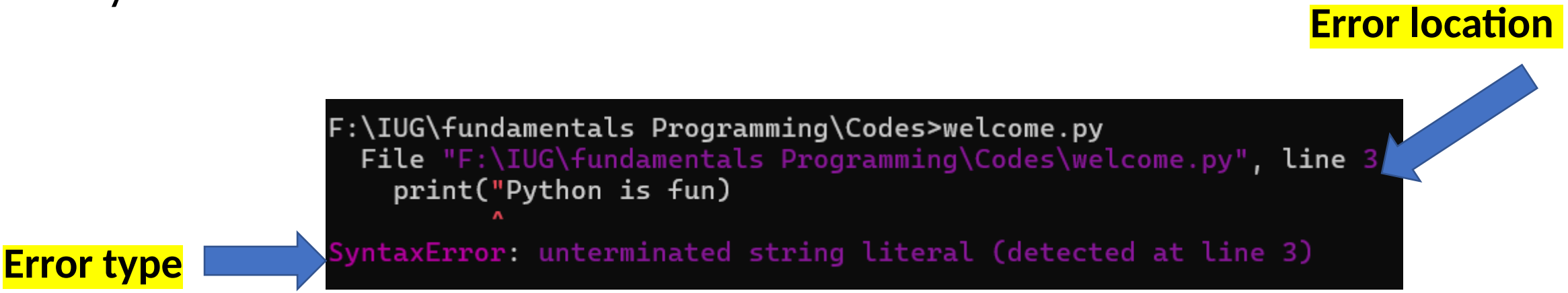
# Syntax Errors

- Occur when code **breaks language rules**
- Violates Python syntax
- **Causes of Syntax Errors**
  - Missing quotation marks
  - Misspelled keywords
  - Incorrect indentation
  - Missing parentheses

```
print("Programming is fun)
```

# Fixing Syntax Errors

- Python shows:
  - Error location
  - Error type
- Easy to detect and fix



# Runtime Errors

- Occur **during program execution**
- Cause program to stop
- **Causes of Runtime Errors**
  - Invalid user input
  - Wrong data type
  - Impossible operations
- **Input Error Example**
  - Program expects a number
  - User enters text
    Causes runtime error

```
print(1 / 0)
```

# Logic Errors

- Program runs **without crashing**
- But produces **incorrect results**
- **Why Logic Errors Are Hard**
  - No error message
  - Program appears correct
  - **Requires** careful testing

# Errors Comparison

| Error Type | When it Happens | Easy to Fix |
|---|---|---|
| Syntax Error | Before run | Yes |
| Runtime Error | During run | Yes |
| Logic Error | After run | No |

# End of Chapter 01